

Module 03: Structured Output, BeanOutputConverter & Type-Safe Extraction

Bridging Probabilistic AI and Deterministic Enterprise Schemas

Instructor: AI Curriculum Engineering Team

Tech Stack: Java 25/27 (LTS), Spring Boot 3.4/4.x, Spring AI 1.0+, Ollama

1. The Non-Deterministic Output Challenge

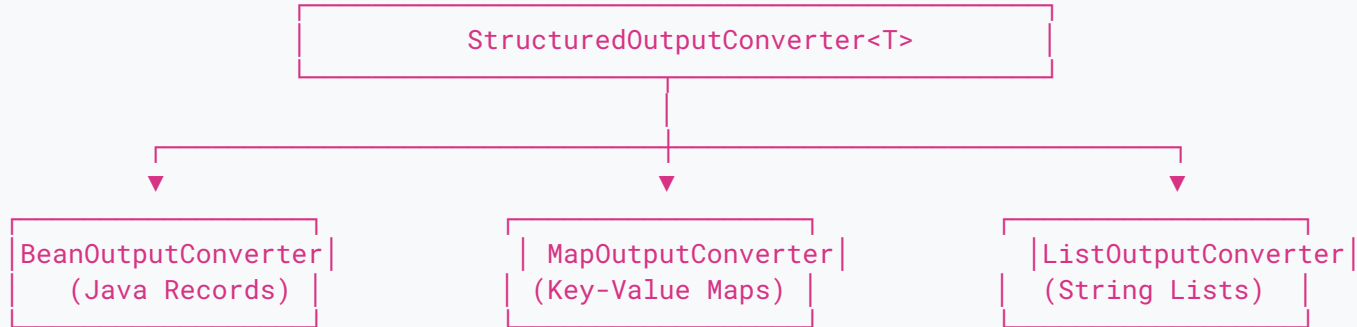
LLMs operate on token probabilities, not relational schemas. Common failures when asking for JSON:

- Wrapping valid JSON in markdown fences (`json ...`).
- Omitting required fields or inventing extra attributes.
- Type hallucinations (e.g., returning `"five"` instead of numeric `5`).

Downstream enterprise services (PostgreSQL, Kafka, REST APIs) cannot tolerate malformed JSON.

2. Spring AI Output Converters

Spring AI provides dedicated converters:



The converter automatically computes the target JSON Schema and injects format directives into the prompt.

3. Defining Schemas with Java Records

Java records provide concise, immutable data models decorated with Jackson annotations:

```
public record CandidateSkill(  
    @JsonPropertyDescription("Name of technical skill")  
    String name,  
  
    @JsonPropertyDescription("Proficiency level: JUNIOR, MID, SENIOR, EXPERT")  
    String proficiencyLevel,  
  
    @JsonPropertyDescription("Years of practical experience")  
    int yearsOfExperience  
) {}
```

Spring AI inspects the record via reflection and generates a rigorous JSON Schema.

4. Fluent ChatClient `.entity()` Extraction

Instead of manual string parsing:

```
@Service
public class ResumeExtractionService {

    private final ChatClient chatClient;

    public CandidateProfile extract(String rawText) {
        return chatClient.prompt()
            .user(u -> u.text("Extract candidate profile from:\n{text}")
                .param("text", rawText))
            .call()
            .entity(CandidateProfile.class); // Directly maps to typed record!
    }
}
```

5. Homework 03 Assignment & Hands-on Lab

"Resume & CV Parsing Service to Java Records"

1. Open `module-03-structured-output/homework/starter`.
2. Inspect `ResumeParserService.java`.
3. Implement candidate extraction using `BeanOutputConverter<ExtractedCandidate>`.
4. Run `mvn test` to verify complete parsing and type validation.